# Snowflake Applied AI: Candidate Homework Assignment (5 pages)

---

## Overview

Your task is to build a production-quality, interactive chat agent that can answer natural language questions about the US population. This is not a prototype exercise — we are evaluating how you approach building something you would be comfortable deploying and handing off to a customer.

**You have 24 hours from the time you receive this assignment to submit your work.**

⚠️ **Note -** *We intentionally leave several technical decisions open-ended. Your choices — and your ability to explain and defend them — are a core part of what we are evaluating.*

**How to prepare:**

- Read the requirements carefully before writing a single line of code. There are specific deliverables — make sure you address all of them.
- Make deliberate architectural choices and be ready to explain them. We are as interested in your reasoning as your output.
- Write your reflection document thoughtfully. It's not an afterthought — it's one of the most important parts of the submission.
- Think about edge cases, failure modes, and what would need to change to take this to production.
- Use AI coding tools freely. We encourage it. Be prepared to discuss how you used them.
- If you run out of time, submit what you have and use the reflection to explain what you would have done with more time.

**What we are looking for**

- The ability to ship something real under time pressure.
- Sound judgment about where to invest your limited time.
- Production-quality thinking: error handling, edge cases, testing.
- Honest, specific self-assessment in your written reflection

---

*Questions: If you have a clarifying question about the requirements, use your best judgement and document your interpretation in your README. This is itself a signal we evaluate.*

---

## Data

Using a free Snowflake trial account, connect to the US Open Census dataset available on the Snowflake Marketplace. This is your required data source. How you query, transform, and serve that data is entirely up to you. If the above data isn't accessible, please use this data: https://www.safegraph.com/free-data/open-census-data/ (if using this data, please make note of it.)

---

## REQUIREMENTS

### Core Functionality

- Build an interactive, chat-based agent that answers natural language questions grounded in the US Census dataset.
- Provide a web-based interface accessible on the public internet (no local setup required to evaluate your submission).
- The agent must preserve conversation context across multiple turns.
- Responses must be returned within 60 seconds under normal conditions.
- Apply guardrails to prevent off-topic or inappropriate responses.

### Production Quality Bar

Your submission should reflect the standards you would apply to code being handed off to a customer.

Specifically:

- The agent must degrade gracefully. If a query cannot be answered given the available data, the agent should say so clearly and helpfully rather than hallucinating, returning an empty response, or throwing an unhandled error.
- Consider and handle ambiguous queries, queries that partially match available data, conflicting or underspecified questions, and queries that are reasonable but unanswerable given the dataset.
- Include meaningful tests that give us confidence in your implementation. You decide the scope and approach — but be prepared to explain your testing strategy and its tradeoffs.
- **Your README should make it easy for a new engineer to understand your architecture and for us to evaluate your running demo.**

💡 **Tip** *We are not expecting perfection across every dimension, given the time constraint. We are evaluating your judgment about where to invest and your self-awareness about the tradeoffs you made.*

## Deliverables

Within 24 hours, email a link to a **private** GitHub repository shared with:

1. sfc-gh-setli
2. sfc-gh-vb
3. Sfc-gh-wenli
4. sfc-gh-cnivera
5. Sfc-gh-twhite
6. Sfc-gh-mtreadwell
7. Sfc-gh-mvytlingam
8. Sfc-gh-cgaboury
9. Sfc-gh-sabbas
10. Sfc-gh-jsummer
11. sfc-gh-htian

The repository must include:

1. All code written to implement your solution.
2. Instructions for accessing your running web application, including credentials if authentication is required.
3. A written reflection (markdown) covering:
    - Your development process and key architectural decisions.
    - What would you improve or do differently with more time?
    - Any edge cases or failure modes you identified but did not fully address.
    - How you approached testing and what you would add to the test suite.

## What We Are Evaluating

We review submissions across four dimensions:

| Dimension | What we look for |
|---|---|
| **LLM / AI Engineering** | Does the solution demonstrate genuine understanding of how to build data-grounded chat agents? Are the architectural choices defensible? |
| **Production Quality** | Is the code something you would ship? How does the agent behave under ambiguous, incomplete, or adversarial inputs? |

| | |
|---|---|
| **Judgment Under Constraints** | Given 24 hours, where did you invest your time? What did you deliberately leave out and why? |
| **Reflection & Self-Awareness** | Does the written reflection show honest assessment of tradeoffs and clear thinking about what could be improved? |

## FAQs

- **Q: Do I need to build and run my solution exclusively on the Snowflake platform?**
  A: No. The data is available through Snowflake Marketplace, but you are free to develop your application however you wish.
- **Q: Is it ok if viewing the demo requires authentication?**
  A: Yes, provided you include credentials in your submission so we can access it.
- **Q: Can I use AI coding tools to help build the solution?**
  A: Yes, and we encourage it. Part of what we are evaluating is how effectively you use the tools available to you. Be prepared to discuss how you used them during the follow-up review.
- **Q: What if I cannot finish everything in 24 hours?**
  A: Submit what you have. Use your written reflection to explain what you would have done with more time. Incomplete submissions that show strong judgment and self-awareness will score better than complete submissions that lack them.

## Tips

### Data Integrity & Grounding

- **Comprehensive Mapping:** Do not limit yourself to a subset. Ensure your system can access the complete range of the provided data, particularly the metadata and join tables that connect different categories together.
- **Context Awareness:** Ensure your agent can "see" enough of the schema to answer nuanced questions without requiring every possible column to be hard-coded into the prompt.

### Functional Reliability

- **The Deployment "Truth":** Your local machine doesn't count. Test your final submission in a clean, deployed environment (like a private URL) to catch missing variables, broken database connections, or authentication loops.

- **Graceful Degradation:** Implement error handling that explains *why* something failed rather than letting the app crash or hang. A descriptive "I'm having trouble connecting to the data" is better than a blank screen.

**Systematic Architecture**

- **Stateful Continuity:** Treat the assignment like a real conversation. The system must remember the context of previous messages to handle follow-up questions accurately.
- **Operational Guardrails:** Implement a dedicated validation layer. This should filter out off-topic requests and ensure that the agent stays within its intended "grounded" knowledge base.

**Performance & UX**

- **The Latency Threshold:** If your process takes more than 60 seconds, it's a failure. Use streaming or progress updates to maintain interactivity and prevent the reviewer from timing out.
- **Safety First:** Prioritize "correct and slow" over "fast and wrong," and aim for a "fast-fail" path in which the agent quickly identifies and rejects unanswerable questions.